

隧道工程智能排水计算平台落地开发路径。遵循可控、可验证的工程实施原则，按开发先后逻辑拆解为以下五个核心执行阶段：

阶段一：工程基座初始化与环境隔离 (第 1 周)

此阶段目标为确立标准化的系统架构，确保前后端及跨端容器物理隔离，互不干扰。

1. **构建 Monorepo 目录：** 新建根目录 `tunnel-drainage-platform` 并初始化 Git 仓库。
2. **初始化后端 (Python)：**
 - 进入根目录，创建 `backend` 文件夹。
 - 配置虚拟环境：`python -m venv venv` 并激活。
 - 安装核心依赖：`pip install fastapi uvicorn pydantic pandas numpy pydantic-settings`。
3. **初始化前端与桌面端 (Vue + Tauri)：**
 - 在根目录执行：`npm create tauri-app@latest`。
 - 按照提示依次选择：`frontend` 作为目录名 -> `Vue` -> `TypeScript`。
 - 进入 `frontend` 目录，安装 3D 与 UI 依赖：`npm install three element-plus pinia axios xlsx`。

阶段二：后端计算逻辑服务化 (第 1-2 周)

此阶段目标为将现有的 Python 计算函数转化为可被第三方或前端调用的独立服务。

1. **数据接口契约化 (Schemas 定义)：** 在 `backend/app/models/schemas.py` 中，使用 Pydantic 严格定义入参。必须显式区分单洞（38个参数）和双洞（41个参数），并将 12 个默认参数直接写死在模型定义中作为保底值。
2. **核心算法接入：** 将您现有的“排水计算模型”原生 Python 文件复制到 `backend/app/services/` 目录下。保持原有函数逻辑不变。重构为 `drainage_engine.py` 核心计算主函数
3. **路由绑定与 API 暴露：** 在 `backend/app/api/endpoints/calculate.py` 中编写 POST 接口，接收前端传入的 JSON 数据，调用 `services` 中的计算函数 `drainage_engine.py`，并组装包含“间距”、“孔径”等优化布置参数的返回体。
4. **单元测试与文档验证：** 本地启动后端 `uvicorn main:app --reload`，访问 `/docs` (Swagger UI)，验证 API 是否能成功接收 38/41 个参数并正确返回计算结果。

阶段三：前端台账式表单与数据流转 (第 2-3 周)

此阶段重点解决重度输入参数的交互体验。

1. **状态机搭建 (Store)：** 配置 Pinia，建立 `parameterStore`，集中存储表单的 38/41 个参数状态。
2. **分区分级 UI 构建：**
 - 使用 Element Plus 的折叠面板组件 (`el-collapse`) 将表单分为：基础输入区、复选区和高级默认区。
 - 预设典型案例数据，实现下拉框一键覆盖 `parameterStore` 中的数值。
3. **Excel 批量 IO 集成：** 引入 `xlsx` 库，开发“下载模板”、“上传解析”按钮，映射至 Store。
4. **快照持久化逻辑：** 开发快照捕获函数，将 Pinia 中的参数状态镜像导出为 JSON 对象并关联时间戳与备注。
5. **前后端联调：** 封装 Axios 请求，在点击“开始计算”时，提取 Store 数据发起 POST 请求，接收并保存排水管网的优化结果。

阶段四：3D 可视化映射与交互开发 (第 3-4 周)

此阶段为核心难点，重点保障大体量构件的渲染性能。

1. **交互状态机开发：** 配置 Pinia 支持参数的“草稿态”与“提交态”，确保拖拽滑块时 3D 视图能实时响应数据变化。
2. **动态构件渲染器：** 重构 Reinforcement.ts，支持注浆圈、衬砌等构件的 UpdateGeometry 方法，实现不销毁对象的实时变形。
3. **计算接口性能优化：** 针对交互场景优化 POST /calculate 接口，剥离非必要校验逻辑，提供高响应的“交互模式”专用计算路由。
4. **画布挂载与基础场景设定：** 在 Vue 组件 (Viewer3D.vue) 中初始化 WebGLRenderer、Scene、PerspectiveCamera，添加环境光与平行光。
5. **低参实例化建模 (隧道主体)：** 开发 TunnelGenerator.ts，接收隧道长度与半径参数，利用 THREE.InstancedMesh 循环生成单环衬砌结构。
6. **计算结果空间映射：** 获取后端的间距与孔径数据。通过数学计算，将排水管沿着隧道管壁的空间坐标计算出来。继续使用 InstancedMesh 生成排水管网，赋予特定的材质（如蓝色反光材质）。
7. **快照 UI 列表与切换：** 在 3D 画布侧边栏开发轻量化快照导航栏，实现点击快照后 3D 构件尺寸与安全系数标签的无缝平滑过渡切换。
8. **事件监听与剖切实现：**
 - 接入 OrbitControls 支持场景视角的自由控制。
 - 配置 THREE.Raycaster，监听鼠标点击事件；击中特定管网网格时，在屏幕空间 (clientX, clientY) 渲染出包含该管间距与孔径信息的 HTML 弹窗。
 - 配置 THREE.Plane 并应用到渲染器的全局裁剪属性，绑定界面的滑块组件，实现剖切动作。

阶段五：系统编译与封装交付 (第 4 周)

此阶段目标为完成独立安装包的输出。

1. **前后端融合配置：** 配置 Tauri 使其能正确代理后端的请求，或在 Tauri 生命周期中打包并拉起 Python 后端进程（对于纯离线需求）。若为网络版应用，直接配置前端请求的 baseUrl。
2. **智能求解算法集成：** 完成“自动搜索临界水头”功能的算法实现与前端 UI 触发开关开发。
3. **交互稳定性测试：** 在重负载数据下测试高频拖拽参数时的系统稳定性，确保内存不溢出、渲染不掉帧。
4. **构建打包：** 执行 npm run tauri build，系统将根据宿主环境自动编译出 Windows (.exe / .msi) 或 macOS (.dmg) 安装包。
5. **性能验收：** 安装桌面包，在 3D 界面导入满载数据（双洞，最长里程），测试渲染帧数与内存占用。